arXiv:2511.10192v4 [cs.CL] 10 Feb 2026

TEXT2SQL-FLOW: A Robust SQL-Aware Data Augmentation Framework for Text-to-SQL

Qifeng Cai†, Hao Liang†, Chang Xu†, Tao Xie, Wentao Zhang*, Bin Cui*
Peking University, Beijing, China
qfcai@stu.ecnu.edu.cn, {hao.liang, cxu25}@stu.pku.edu.cn,
{taoxie, wentao.zhang, bin.cui}@pku.edu.cn

Abstract*—The data-centric paradigm has emerged as a pivotal direction in artificial intelligence (AI), relying on high-quality training data. This shift is especially critical in the Text-to-SQL task, where the scarcity, limited diversity, and structural simplicity of existing datasets constrain model performance. To address these challenges, we propose TEXT2SQL-FLOW, a SQL-aware data augmentation framework that systematically generates large-scale, semantically valid, and structurally diverse Text-to-SQL pairs from limited seed data. Our framework operates along six augmentation dimensions and integrates an end-to-end pipeline featuring auxiliary database selection, SQL executability verification, natural language (NL) question generation, NL–SQL correspondence verification, and chain-of-thought (CoT) reasoning trace generation. Leveraging this framework, we construct SQLFLOW, a high-quality dataset comprising 75,386 annotated examples. We demonstrate the utility of SQLFLOW in both fine-tuning and prompt-based settings: (1) For open-source large language models (LLMs), fine-tuning with SQLFLOW enhances the problem-solving capabilities. Under the same data budget, models trained on SQLFLOW achieve competitive performance gains across multiple benchmarks. (2) For closed-source LLMs, we propose a masked alignment retrieval method that leverages SQLFLOW as both a knowledge base and the training data for the retrieval model. This approach enables structure-aware example matching by modeling fine-grained alignments between NL questions and SQL queries. Experimental results show that our retrieval strategy outperforms existing example retrieval methods, highlighting the dual importance of SQLFLOW's high-quality data and our novel retrieval technique. Our work establishes a scalable, data-centric foundation for advancing Text-to-SQL systems and underscores the indispensable role of structured, high-fidelity data in modern AI development. Our code is available at https://github.com/TechNomad-ds/Text2SQL-Flow.

***Index Terms*—Text-to-SQL, Large Language Model, SQL**

I. INTRODUCTION

In recent years, the data-centric artificial intelligence (AI) paradigm has garnered increasing attention [1], [2]. Unlike traditional algorithm-centric approaches that primarily focus on advancing model architectures and algorithms, many AI applications are increasingly constrained by the availability of high-quality data. Vast amounts of data remain underutilized, despite their immense potential value. From a data-centric perspective, high-quality, diverse, and well-structured training data plays a central role in advancing AI [3].

The Text-to-SQL task is currently facing an urgent demand for high-quality data. Although the structured query language (SQL) enables powerful and precise database interactions, its complex syntax and steep learning curve limit its accessibility for non-expert users. The Text-to-SQL task aims to translate natural language (NL) questions into executable structured query language (SQL) queries, allowing users without specialized knowledge to access databases conveniently [4]. This capability shows broad application prospects in intelligent analysis, business intelligence platforms, and enterprise data services. With the rise of LLMs, significant progress has been made in Text-to-SQL research.

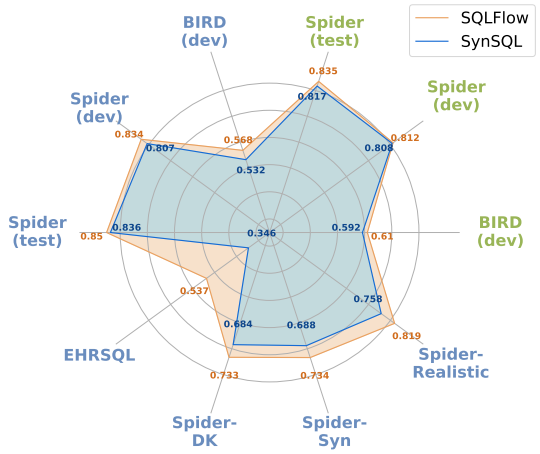

Fig. 1: Performance comparison of models trained on SQLFlow vs. SynSQL [5] (both at 75,386 samples). Blue labels indicate results for fine-tuned open-source models, while green labels indicate results of closed-source model with few-shot retrieval for prompt construction.

Existing methods are mainly divided into two categories: *(1) fine-tuning-based methods* [6]–[9] and *(2) prompt-based methods* [10]–[13]. In both categories of methods, high-quality data plays a pivotal role. Fine-tuning-based methods rely on data pairs to enable LLMs to learn the mapping from NL questions to SQL queries, thereby enhancing the model's reasoning capability. In prompt-based methods, where model weights are inaccessible, in-context learning becomes key to improving performance, and the quality of retrieved few-shot examples is critical. Data can serve as reference examples in a knowledge base or be used to train retrieval models for obtaining more representative few-shot samples [6], [14], [15].

†Equal contribution. *Corresponding authors.

TABLE I: Comparison of SQL query complexity between the original seed datasets and the datasets augmented by TEXT2SQL-FLOW. Rows prefixed with "SQLFlow-" show augmented dataset stats; the bottom row shows overall SQLFLOW statistics.

Dataset	Size	Query Feature Presence per SQL %				Feature Count per SQL		
		Window Func.	Set Op.	Subquery	GROUP BY	# CASE	# WHERE	# JOIN
Spider-train	8,659	0.00	6.07	15.59	22.60	0.00	0.86	0.71
SQLFlow-Spider	30,502	14.37	9.49	40.06	37.67	0.22	1.84	0.96
BIRD-train	9,428	0.04	0.23	6.71	9.16	0.09	1.25	0.90
SQLFlow-BIRD	33,163	12.39	7.06	36.28	29.86	0.26	2.36	1.10
EHRSQL-train	18,472	11.99	0.23	67.72	20.72	0.02	4.13	0.42
SQLFlow-EHRSQL	11,721	21.65	7.47	66.86	34.14	0.16	4.32	0.95
SQLFLOW	75,386	14.63	8.11	42.57	33.68	0.23	2.46	1.02

Since the Text-to-SQL task relies on NL questions and corresponding SQL queries, efficiently acquiring large-scale, high-quality data remains a fundamental challenge. Manual collection not only struggles to guarantee quality but is also limited in scale. Existing public datasets (e.g., Spider [16], BIRD [17]) mainly rely on manual annotation, which leads to limited size and narrow data coverage. Recent studies [5], [18] have explored the potential of automatically synthesizing Text-to-SQL datasets at scale using LLMs. However, these approaches often overlook the rich structural and semantic information already embedded in existing curated Text-to-SQL datasets. These datasets, which are typically carefully curated and execution-verified, provide a strong foundation for systematic and scalable data expansion. Failing to fully leverage such high-quality resources may result in suboptimal data efficiency and unnecessary redundancy in data generation.

Data diversity in Text-to-SQL can be characterized along two dimensions. *Distribution-level diversity* refers to variations in underlying database schemas, data domains, and result distributions, and is typically introduced by incorporating databases from different sources or domains. In contrast, *query-level diversity* captures the structural and compositional variations of SQL queries, such as aggregation patterns, join complexity, nested subqueries, and logical constraints.

To address these challenges, we propose TEXT2SQL-FLOW, a robust and scalable Text-to-SQL data augmentation framework that systematically expands existing seed data derived from original databases and their corresponding SQL queries. To enhance distribution-level diversity, our framework first introduces auxiliary databases by selecting databases with varying structural complexities and data domains, thereby enriching the overall data landscape. Building upon seed SQL queries from both original and auxiliary databases, TEXT2SQL-FLOW defines six data augmentation dimensions spanning data value transformations, query structure modifications, and complexity enhancements. Leveraging LLMs, the framework synthesizes diverse and semantically valid augmented SQL queries. A series of specialized operators is then employed to perform SQL executability filtering, NL question generation, NL–SQL alignment filtering, prompt construction, and chain-of-thought (CoT) reasoning path generation. These operators collaboratively ensure the quality, correctness, and diversity of the generated data. Through the combination of LLM-driven generation and automated filtering mechanisms, TEXT2SQL-FLOW is capable of producing large-scale, high-quality, and diverse Text-to-SQL datasets from limited original data, while maintaining strong scalability and extensibility across heterogeneous databases. Furthermore, a unified database interaction layer is adopted to abstract database-specific details, enabling efficient SQL execution and schema access across multiple relational database systems.

Based on our SQL augmentation pipeline, a high-quality Text-to-SQL dataset comprising 75,386 entries called SQLFLOW is constructed. Using SQLFLOW, we explore the applications of data in Text-to-SQL tasks. For open-source LLMs, the augmentation framework can provide higher-quality training data. The included CoT can further assist models in improving their problem-solving capabilities. After fine-tuning open-source models with data generated by our framework, we achieved excellent performance on multiple benchmarks, validating the framework's effectiveness. As shown in Figure 1, under the same data scale, the model trained by SQLFLOW achieves higher performance than SynSQL [5] across all benchmarks, attaining 56.8% on the BIRD-dev dataset. For closed-source models, we adopt prompt-based methods, retrieving few-shot examples to provide references for the LLM in solving problems, thereby achieving broad applicability across various scenarios. Existing systems typically select examples from limited public datasets, which have significant limitations in semantic diversity and structural coverage. Therefore, the knowledge base for retrieval can also be constructed using the data generated by our framework. However, at the retrieval method level, current mainstream retrieval strategies often focus merely on surface-level semantic similarity, neglecting the deep syntactic and logical relationships between NL questions and SQL queries. To address this, we propose an improved masked alignment retrieval method for example selection. This method learns the masked alignment relationships between NL questions and SQL queries during the training phase and achieves fine-grained structure-aware matching during the inference phase, enhancing the relevance and effectiveness of retrieved

examples. The training data for this retrieval model also utilizes the data constructed by our framework. As shown in Figure 1 shows that our retrieval method achieves competitive performance on benchmarks, with 61.0% on the BIRD-dev and 83.5% on the Spider-dev, outperforming other methods.

Our contributions are summarized as follows:

- We propose TEXT2SQL-FLOW, a robust Text-to-SQL data augmentation framework that uses existing Text-to-SQL pairs as seeds to systematically generate semantically valid and structurally diverse augmented samples along six dimensions efficiently. An end-to-end automated pipeline is designed, integrating modules for auxiliary database selection, SQL augmentation generation, SQL executability filter, NL question generation, NL-SQL alignmentation filter, prompt generation, and CoT generation, ensuring high-quality generated data.
- Based on TEXT2SQL-FLOW, we generate a high-quality Text-to-SQL dataset named SQLFLOW, comprising 75,386 samples. The data is efficiently generated, exhibiting high diversity and quality. Experiments on open-source LLMs show that fine-tuning with SQLFLOW enhances model performance, achieving competitive results efficiently on benchmark evaluations. Notably, it attained 61.0% on the BIRD-dev dataset and 85.1% on the Spider-test dataset, outperforming other models trained with datasets of similar scale.
- For scenarios involving closed-source models, we utilize SQLFLOW as a few-shot example knowledge base and further propose a masked alignment retrieval method. By learning fine-grained alignment relationships between NL questions and SQL queries, our method enables more accurate example matching. Experiments show that our retrieval model, trained using SQLFLOW, enhances the Text-to-SQL accuracy of closed-source models, outperforming existing retrieval methods based on semantic similarity, achieving 61.0% on the BIRD-dev dataset and 83.5% on the Spider-dev dataset.

II. RELATED WORKS

A. Text-to-SQL Techniques

Early rule-based approaches map NL questions to SQL using handcrafted templates [19]–[21], but their reliance on rigid rules severely limits generalization across domains and schemas. Neural sequence-to-sequence models [22]–[25] introduce encoder–decoder architectures with attention and schema-aware representations to model the alignment between NL and database structures. However, these methods still struggle with robustness when generalizing to unseen or heterogeneous schemas. Pre-trained language models such as BERT [26], [27] and T5 [28] further improve Text-to-SQL by enhancing semantic alignment between NL utterances and database schemas [29], [30]. Nonetheless, their performance heavily depends on large amounts of annotated data and degrades in low-resource or cross-domain settings.

Recently, the emergence of LLMs has opened new directions for Text-to-SQL. Through sophisticated prompting strategies, LLMs can generate accurate SQL queries even in zero-shot or few-shot settings [14], [31]. Techniques including prompt engineering [6], task decomposition [13], and self-correction have further enhanced their reliability and generalization capabilities, demonstrating considerable promise for cross-domain and real-world applications.

B. Data-Centric AI for Text-to-SQL

Data-centric approaches for Text-to-SQL focus on leveraging existing data and synthetic data generation. For existing data, prior work improves schema understanding via schema linking and structural representations [12], [32]–[34], along with preprocessing and cleaning techniques [7], [35]–[37] that refine schema information and inter-table relationships.

Synthetic data generation methods are generally divided into SQL-to-question and question-to-SQL paradigms. SQL-to-question approaches [38]–[41] generate natural language questions from SQL queries using templates or rules, producing high-fidelity pairs but suffering from limited scalability and difficulty in handling complex queries. Question-to-SQL methods [10], [42]–[44] generate questions first and then infer SQL using Text-to-SQL models; while more natural, these methods are prone to error propagation. Recent work seeks to mitigate these issues. Sense [18] adopts prompt chaining to jointly generate schema, question, and SQL with semantic consistency, but relies on costly closed-source models. OmniSQL [5] instead decouples the generation pipeline and leverages open-source or lightweight LLMs with CoT reasoning, offering a more scalable and reproducible solution.

III. METHODOLOGY

Despite the growing interest in the Text-to-SQL task, its development is fundamentally constrained by the limited availability of high-quality SQL data. For a given Text-to-SQL system, the user typically provides an existing database d_o, which serves as both the available database resource and the target database on which the Text-to-SQL task is performed. Although there exists a set of manually annotated SQL queries $\mathcal{Q}_o$ over d_o, its scale is usually limited. The objective is therefore to improve the model's performance on the Text-to-SQL task over d_o. However, due to the high cost of expert annotation, $\mathcal{Q}_o$ is small in scale, making it insufficient to rely solely on it for learning robust and generalizable query patterns. Nevertheless, $\mathcal{Q}_o$ is typically of high quality, with accurate semantics and strong alignment to the underlying database schema, making it a reliable foundation for data expansion. As a result, how to effectively leverage $\mathcal{Q}_o$ to construct large-scale and diverse SQL data remains challenging.

When expanding the scale of SQL data, diversity should not be overlooked. SQL diversity can be characterized along two complementary dimensions. *Distribution-level diversity* captures variations in database schemas, data domains, and underlying data distributions. Such diversity enhances domain coverage and mitigates overfitting to database-specific patterns. Meanwhile, *query-level diversity* reflects structural and compositional variations of SQL queries. Both dimensions are

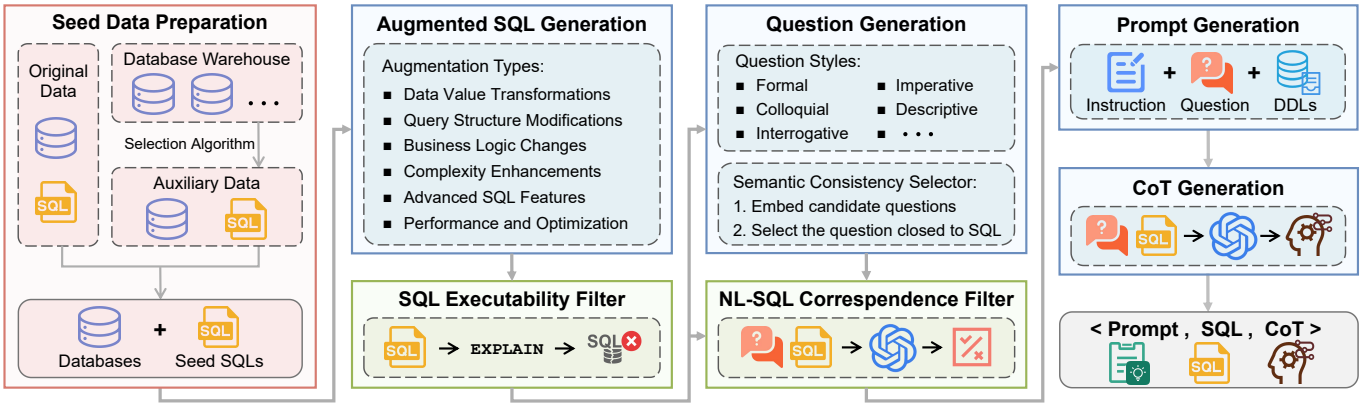

Fig. 2: Overall framework of our work.

crucial for constructing high-quality augmented SQL datasets. Motivated by these insights, we propose a joint data augmentation framework, TEXT2SQL-FLOW (shown in Figure 2), which simultaneously enhances distribution-level diversity and query-level diversity under data-scarce settings. Based on this framework, we further design a data generation pipeline to construct large-scale, high-quality, and diverse SQL data.

A. Distribution-level Diversity Augmentation

Distribution-level diversity captures variations in database schemas, data domains, and underlying data distributions. When the original database d_o is fixed, an effective way to improve distribution-level diversity is to introduce additional databases as auxiliary resources. Each auxiliary database is typically accompanied by its own set of existing SQL queries, which naturally expands the diversity of SQL patterns. Moreover, newly introduced databases bring novel data domains and distinct data distributions, thereby enriching the space of feasible SQL augmentations. This is because SQL queries are inherently shaped by the underlying data distributions they operate on. However, the number of available databases is often large, making it impractical to incorporate all of them. We therefore construct a database warehouse $\mathcal{D}_w$ consisting of publicly available and accessible databases. Each database $d \in \mathcal{D}_w$ is associated with a corresponding SQL set $\mathcal{Q}(d)$. Given resource constraints, it is infeasible to utilize the entire warehouse. Instead, we select a subset of databases from $\mathcal{D}_w$ as auxiliary databases, denoted as $\mathcal{D}_a$, to enhance distribution-level diversity. Our key principle for selecting $\mathcal{D}_a$ is that, under a limited budget, the chosen auxiliary databases should cover a wide range of schema complexities.

For each candidate database $d \in \mathcal{D}_w$, we compute a set of complexity metrics that depend solely on the database schema structure, including the number of tables $T(d)$, the total number of columns $C(d)$, and the foreign key density $\text{FK\_density}(d) = \frac{\text{FK}(d)}{\max(T(d),\epsilon)}$ where ϵ is a small smoothing constant used to ensure numerical stability when the number of tables is small. To avoid inconsistencies in scale across different metrics and the need for manually specified weights, we compute the percentile rank of each complexity metric over the database warehouse $\mathcal{D}_w$. Specifically, for any metric $x(d)$, its percentile rank is defined as $r_x(d) = \frac{1}{|\mathcal{D}_w|}\sum_{d' \in \mathcal{D}_w} \mathbb{I}[x(d') \leq x(d)]$ where $\mathbb{I}[\cdot]$ denotes the indicator function. Accordingly, we obtain $r_T(d)$, $r_C(d)$, and $r_{FK}(d)$. Based on these rankings, we define the schema complexity score of a database as

$$\text{score}(d) = r_T(d) + r_C(d) + r_{FK}(d). \tag{1}$$

We then sort all candidate databases according to $\text{score}(d)$ and select k quantile points at uniform intervals over the score distribution, thereby determining k auxiliary databases.

We then construct a combined corpus consisting of the SQL queries from the original database $\mathcal{Q}_o$ and those from the auxiliary databases $\bigcup_{d \in \mathcal{D}_a} \mathcal{Q}(d)$. Although this process substantially increases distribution-level diversity, the SQL queries themselves remain constrained by the structural patterns inherent in the original datasets. Therefore, an augmentation strategy over existing SQL queries is required to further expand query-level diversity.

B. Query-level Diversity Augmentation

*1) **SQL Augmentation**:* Existing approaches to SQL data synthesis often focus on generating queries from scratch, such as via templates or random construction, overlooking the untapped potential of high-quality SQL queries already available. This limits the effective utilization of existing resources. In contrast, such high-quality queries can be repurposed through data augmentation to generate diverse yet semantically equivalent or closely related augmented SQL data. Moreover, SQL queries are inherently tied to their underlying database schemas. Constraints such as table or column names and foreign key relationships are implicitly encoded in valid SQL queries. Augmenting from existing queries naturally preserves schema consistency, ensuring that the generated augmented data remains syntactically valid and semantically aligned with the original data distribution. This yields augmented data that better reflects real-world query patterns while maintaining fidelity to the database schema.

A key challenge in data augmentation lies in producing meaningful and diverse augmented data from existing SQL queries. Generating schema-consistent and semantically coherent queries not only enriches the training data but also enhances the model's robustness and adaptability to a wide range of query formulations. To this end, we propose a data augmentation framework that leverages LLMs to generate high-quality augmented SQL data. Thanks to extensive exposure to SQL snippets during the pre-training process, LLMs have acquired strong priors over SQL syntax, semantics, and common query patterns, making them well-suited for generating syntactically valid and semantically faithful SQL data. However, a critical limitation remains: existing public SQL datasets often exhibit limited query patterns and relatively low structural complexity. When used as prompts or seeds, such simplistic queries can constrain the diversity and sophistication of the generated variants, hindering coverage of real-world usage scenarios. To overcome this, we design a set of targeted augmentation strategies that preserve the core semantics of the original queries while systematically increasing their structural complexity and stylistic variation. This enables the construction of a more comprehensive and representative Text-to-SQL dataset. Six augmentation strategies are proposed to generate SQL queries in different directions, which can be summarized as follows:

- **Data Value Transformations (DVT)**: Modify query parameters to refine result precision, including filtering conditions, date ranges, numeric thresholds, sorting criteria, limit values, and aggregation boundaries (e.g., `GROUP BY` at different temporal granularities).
- **Query Structure Modifications (QSM)**: Alter the structural form of queries by rewriting aggregation queries as window functions (and vice versa), introducing subqueries or common table expressions (CTEs), substituting `JOIN` with `EXISTS` or `IN`, and switching between correlated and uncorrelated subqueries to improve flexibility and maintainability.
- **Business Logic Changes (BLC)**: Adapt queries to alternative analytical objectives or business contexts, such as shifting across domains (e.g., sales to inventory), adjusting data granularity (e.g., daily to monthly), changing analytical perspectives (e.g., profit to cost), or modifying evaluation metrics (e.g., sum to average).
- **Complexity Enhancements (CE)**: Increase query complexity by introducing additional filtering conditions, joining extra tables, incorporating `CASE` expressions for conditional logic, or embedding data validation and quality checks.
- **Advanced SQL Features (ASF)**: Employ advanced SQL constructs, including partitioned window functions, set operations (`UNION`, `INTERSECT`, `EXCEPT`), recursive CTEs, and pivot/unpivot operations, to extend the technical expressiveness of queries.
- **Performance and Optimization (PO)**: Optimize query execution by applying optimization hints, restructuring queries to leverage indexes, adopting efficient query patterns, and refining `WHERE` clauses to balance performance and resource utilization.

To ensure the validity of the generated augmented SQL queries, we incorporate randomly sampled database values into the prompts to improve semantic grounding. Each prompt consists of the following components: *(1) Instruction* (I_{sql}): a directive guiding the LLM to generate SQL queries that satisfy realistic analytical requirements; *(2) Database Schema* (S): the full set of `CREATE TABLE` statements describing all relations in the database; *(3) Database Values* (V): Values are randomly sampled from multiple rows in each table to construct meaningful query predicates; any sampled row containing null values is replaced. *(4) Original SQL query* (s_i^{orig}): the query to be augmented; and *(5) Augmentation Direction* (c): one of six predefined augmentation strategies. Let $C = \{c_1, \ldots, c_6\}$ denote the set of augmentation strategies. For each original query s_i^{orig}, we uniformly sample a strategy $c \sim \text{Uniform}(C)$, draw representative database values V from the relevant tables, and construct an augmentation prompt $P_{\text{aug}}(I, S, s_i^{\text{orig}}, V, c)$. The augmented query is then produced by the LLM as:

$$s_i^{\text{aug}} = \text{LLM}\big(P_{\text{aug}}(I_{\text{sql}}, S, V, c, s_i^{\text{orig}})\big) \tag{2}$$

where $P_{\text{aug}}(\cdot)$ denotes the prompt construction process, and $\text{LLM}(\cdot)$ represents the stochastic query generation by LLM.

*2) **SQL Executability Filter**:* Not all generated SQL queries are executable, as some may contain syntax errors or reference non-existent database objects. Therefore, we apply an SQL executability filter to remove invalid candidates. Under a fixed database schema and a read-only setting without user-defined functions, we verify executability at the plan level. Specifically, for each generated SQL query, we check whether the database can successfully parse the query and generate an execution plan using `EXPLAIN` (e.g., `EXPLAIN QUERY PLAN` in SQLite). A query is retained if plan generation succeeds; otherwise, it is discarded. Since this verification does not execute the query, it is insensitive to runtime variability such as cold caches or transient system load. This design choice avoids unintentionally biasing the augmented SQL set toward "fast" queries, while ensuring that all retained queries are logically executable under the fixed schema, which is sufficient for our SQL data augmentation task.

*3) **Question Generation**:* After obtaining the valid augmented SQL queries, the next step is to generate semantically equivalent NL questions. To ensure linguistic diversity in the synthetic data, it is essential to incorporate a wide range of stylistic instructions. Inspired by OmniSQL [5], we analyze real-world user queries along multiple linguistic dimensions and identify eleven common stylistic categories, grouped into four high-level aspects:

- **Tone and Formality (TF)**: levels of formality, including formal and colloquial styles
- **Syntactic Structure and Intent (SSI)**: sentence types such as imperative, interrogative, and declarative
- **Information Density and Clarity (IDC)**: degrees of conciseness, descriptiveness, vagueness, and metaphorical expression

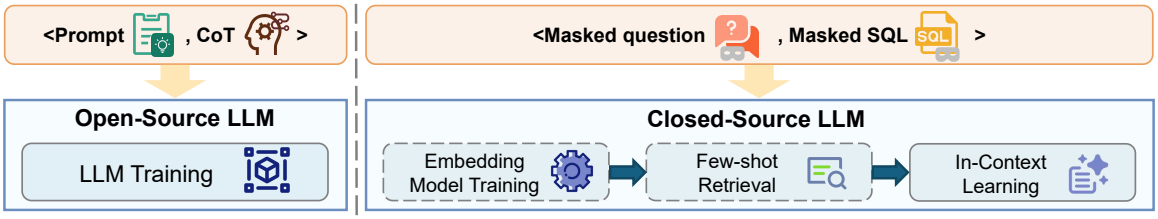

Fig. 3: Augmented data utilization for open-source and closed-source LLMs

- **Interaction Patterns (IP)**: interaction modes such as role-playing and procedural interactions

The first two categories capture cases where user intent is unambiguous but expressed through varying tones or syntactic forms. In contrast, vague and metaphorical styles involve ambiguous or figurative language, often requiring external or contextual knowledge for accurate interpretation.

To synthesize NL questions, we design structured prompts for LLMs comprising the following four components: *(1) Instruction* (I_{nl}): a directive instructing the LLM to translate a given SQL query into an NL question; *(2) Augmented SQL Query* (s_i^{aug}): the SQL query to be translated; *(3) Database Schema* (S): the complete schema provided as `CREATE TABLE` statements; and *(4) Target Language Style* (τ): a stylistic variant randomly sampled from a predefined set $\mathcal{T} = \{\tau_1, \ldots, \tau_{11}\}$, where each style is defined with a description and illustrative examples.

For each SQL query s_i, we sample a style $\tau \sim \text{Uniform}(\mathcal{T})$ and assemble the prompt as $P_{\text{nl}}(I, s_i^{\text{aug}}, S, \tau)$. The LLM then generates an NL question:

$$q_i = \text{LLM}\big(P_{\text{nl}}(I_{\text{nl}}, S, \tau, s_i^{\text{aug}})\big). \tag{3}$$

To ensure both semantic fidelity and linguistic diversity, we generate k candidate questions $\{q_{i1}, \ldots, q_{ik}\}$ for each s_i^{aug} by independently sampling a new style τ for each generation. Among these candidates, we select the candidate question that maximizes semantic consistency with the underlying SQL query, while maintaining agreement across stylistic variants. The final synthetic dataset is constructed as $\mathcal{D} = \{(q_i, s_i^{\text{aug}})\}_{i=1}^{N}$, where N is the number of distinct SQL queries. This approach yields a dataset with rich linguistic variation and broad semantic coverage, enhancing both robustness and generalization in downstream applications.

*4) **NL–SQL Correspondence Filter**:* Even when an SQL query is executable, it may not faithfully reflect the intent of its paired NL question. To ensure semantic consistency between the NL question and the corresponding SQL query, we introduce a NL–SQL correspondence filter. Specifically, given a database schema S, a natural language question q_i, and its augmented SQL query s_i^{aug}, we employ LLM to assess whether the two express the same query intent:

$$\mathbb{I}_{\text{align}}(S, s_i^{\text{aug}}, q_i) = \mathbb{I}_{\text{LLM}}(S, s_i^{\text{aug}}, q_i), \tag{4}$$

where $\mathbb{I}_{\text{LLM}}(\cdot) \in \{0, 1\}$ indicates whether semantic equivalence is confirmed. Only question–SQL pairs with $\mathbb{I}_{\text{align}} = 1$ are retained for subsequent use.

*5) **Prompt Generation**:* For open-source LLMs, problem-solving capabilities can be enhanced through supervised fine-tuning (SFT), a process in which the model learns to map prompt inputs p_i to the corresponding outputs. To facilitate reliable Text-to-SQL generation, a well-structured prompt (p_i) incorporates not only the NL question (q_i) but also the database schema (S) and a specific task instruction (I_{prop}) that guides the model. The process of prompt construction is formally defined as follows:

$$p_i = P_{\text{prop}}(I_{\text{prop}}, S, q_i) \tag{5}$$

where $P_{\text{prop}}(\cdot)$ denotes the function that synthesizes the final prompt from its constituent components.

*6) **Chain-of-Thought Generation**:* To train high-quality LLMs capable of solving complex problems, supervising models using only the final SQL outputs is often insufficient. Chain-of-thought (CoT) traces generated by stronger models decompose the question-solving process into a sequence of smaller, manageable subproblems, enabling step-by-step reasoning. Such CoT supervision provides informative learning signals and can be leveraged to improve the overall reasoning performance of weaker LLMs. In the context of Text-to-SQL translation, CoT traces typically involve the following steps: (1) analyzing the intent of the NL question, (2) interpreting the database schema, (3) identifying relevant tables and columns, (4) filtering necessary information, (5) selecting appropriate SQL operators, and (6) incrementally constructing the SQL.

To generate CoT reasoning traces, we employ an LLM with strong reasoning capabilities. The prompt design includes *(1) an instruction* (I_{cot}), *(2) the database schema* (S), *(3) the generated NL question* (q_i), and *(4) the augmented SQL query* (s_i^{aug}). Guided by this prompt, the LLM produces a complete reasoning chain that encompasses intermediate reasoning steps as well as the final SQL query, which can be expressed as:

$$cot_i = \text{LLM}\big(P_{\text{cot}}(I_{\text{cot}}, S, s_i^{\text{aug}}, q_i)\big) \tag{6}$$

where $P_{\text{cot}}(\cdot)$ denotes the CoT prompt construction process. During validation, the generated SQL query is extracted from the reasoning chain. A CoT trace is accepted as a valid solution only if the execution result of the generated SQL matches that of the reference SQL on the given database.

*7) **Augmented Data Overview**:* Leveraging the proposed Text-to-SQL data augmentation pipeline, we generate an augmented dataset from the original SQL-query pairs s_i^{ori}. The pipeline systematically synthesizes diverse and valid Text-to-SQL examples, which can serve as a valuable supplement to

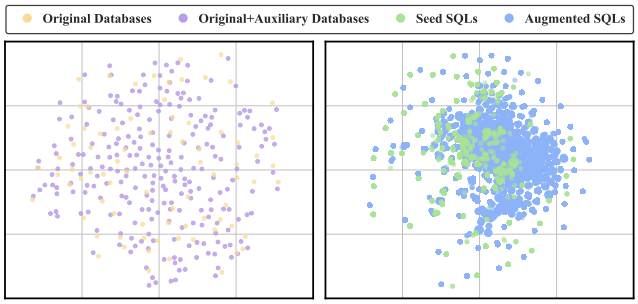

Fig. 4: Visualization of changes in distribution-level (left) and query-level (right) diversity induced by data augmentation.

existing Text-to-SQL datasets. The final dataset comprises N entries of the following form:

$$\left\{s_i^{\text{aug}},\, q_i,\, S,\, p_i,\, \text{cot}_i\right\}_{i=1}^{N}$$

Using TEXT2SQL-FLOW, we construct a dataset named SQLFLOW. The raw seed data is sourced from widely used benchmarks. The final dataset comprises a total of 75,386 entries: 30,502 derived from the Spider-train dataset [16], 33,163 from the BIRD-train dataset [17], and 11,721 from the EHRSQL-train dataset [45]. In this paper, we refer to these subsets as SQLFlow-Spider, SQLFlow-BIRD, and SQLFlow-EHRSQL, respectively. It should be noted that the augmented data do not overlap with the original training datasets. In addition, an `n-gram`-based filtering strategy is applied to ensure that SQLFLOW has no overlap with the test data, thereby preventing any potential data leakage. Overall, our approach augments existing public datasets and is scalable, efficient, and capable of generating high-quality synthetic data. As shown in Table I, our pipeline not only mitigates the sparsity and structural gaps inherent in the original dataset but also substantially extends its complexity frontier—surpassing the original in both scale and quality. The broader coverage and increased diversity enabled by data augmentation are reflected in Figure 4. Using a 2D t-SNE projection, the figure illustrates both database-level and feature-level distributions. The left panel indicates that auxiliary databases expand the coverage beyond the original dataset, while the right panel reveals a more diverse spread in feature representations derived from the statistics in Table I.

C. Database Interaction Implementation

Multiple stages of the TEXT2SQL-FLOW data augmentation pipeline require frequent interactions with the underlying database, including executing SQL queries for validity checking and accessing schema information for query generation. To support multiple database systems and improve implementation robustness, we implement a database interaction layer that abstracts database-specific driver details and provides a unified interface to upper-layer components. This interface supports batched SQL execution and schema access through standardized APIs, enabling seamless integration of different relational database systems (e.g., SQLite, MySQL) without modifying the core pipeline logic. For efficiency, schema information is cached after the first access to avoid redundant metadata queries during large-scale SQL generation.

D. Synthetic Data for Open-Source Models

The pairs $\langle p_i, cot_i \rangle$ generated above serve as high-quality training data $\mathcal{D}$. During SFT, the model is trained on the full output sequence using standard causal language modeling:

$$\mathcal{L} = -\mathbb{E}_{(x,y)\sim\mathcal{D}}\left[\sum_{t=1}^{|y|}\log P_\theta(y_t \mid x, y_{<t})\right] \tag{7}$$

where the input x corresponds to the prompt p_i, and the output y represents the complete CoT sequence cot_i. The model parameters are denoted by θ. By optimizing this loss over the entire output sequence, the model is encouraged to learn interpretable reasoning pathways and improve its performance on the Text-to-SQL task.

E. Synthetic Data for Prompting-based Methods

For closed-source LLMs, whose parameters are not accessible, prompt-based methods have become a crucial technique for improving performance. A typical prompt provided to an LLM generally includes multiple components: an instruction, database schema information, and the NL question posed by the user. To enhance the model's generalization ability across different questions, few-shot examples are often retrieved and incorporated into the prompt context in practice. These examples are typically selected due to their similarity to the target question, serving as guidance for the model in generating the corresponding SQL query. Each few-shot example consists of an NL question paired with its corresponding SQL query. Ideally, such examples should effectively reveal the structural patterns and generation logic of the target SQL, thereby improving the model's reasoning performance.

Two problems need to be considered during few-shot example retrieval: *(1) What information is available during retrieval?* During retrieval, only the user's NL question is available, and we cannot know the ground-truth SQL query in advance. Moreover, any intermediate representation derived from this NL question may introduce information loss, thereby degrading retrieval quality. *(2) Which kind of examples do we need?* The primary value of few-shot examples lies in providing structural guidance for SQL generation. Therefore, the SQL queries of the retrieved examples should exhibit high similarity to the target query, but not necessarily the higher the similarity, the better. Notably, different components of the NL question and SQL query contribute unequally to the similarity. Specifically, the "skeleton" of a question reflects its basic querying pattern, while the skeleton of an SQL query manifests as the logical composition of core clauses such as "SELECT–FROM–WHERE–JOIN". These skeletal elements encode the query's logical intent and functional structure, serving as critical signals for guiding model generation. In contrast, fine-grained details, such as specific table names, column names, string literals, or numeric constants, are highly

dependent on the particular database schema and application context. If directly used in similarity computation, they often introduce noise and impair retrieval effectiveness. Thus, an ideal retrieval mechanism should identify examples whose corresponding SQL queries are structurally aligned with the target query, based solely on the NL question.

However, existing methods for retrieving few-shot examples fail to provide effective guidance because similarity has not been sufficiently exploited. In this case, we propose a structure-aware few-shot retrieval method for Text-to-SQL. The core idea is to map both NL questions and SQL queries into a unified, detail-abstracted semantic space, where retrieval is performed based on structural similarity. Specifically, inspired by the masking strategy in DAIL–SQL [6], we apply standardized masking to schema-dependent elements (e.g., table/column names), string literals, and numeric constants in both questions and SQL queries, denoted as $\text{Mask}(q)$ and $\text{Mask}(\text{s})$, respectively. Building upon this, we develop a structure-aware embedding model to compute semantic similarity between masked NL questions and masked SQL queries. Given a target question q_t, we retrieve the top-k examples from a retrieval pool $B = \{(q_i, s_i)\}_{i=1}^{N}$, which we refer to as a knowledge base. B consists of NL–SQL pairs from the training dataset, with no overlap with the test dataset. B is stored as an unordered collection of independent data pairs, rather than a structured knowledge graph.

We fine-tune our embedding model using SFT and employ a contrastive learning framework for training. The training data consist of numerous (q_i, s_i) pairs. For each sample, the NL question q_i serves as the query, its corresponding SQL s_i as the positive example, and k randomly sampled SQL queries from the knowledge base (excluding s_i) as negative examples. The model is trained by optimizing the InfoNCE loss:

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(q, s^+)/\tau)}{\exp(\text{sim}(q, s^+)/\tau) + \sum_j \exp(\text{sim}(q, s_j^-)/\tau)} \quad (8)$$

where $\mathbf{e}_q^i = \text{Embed}(\text{Mask}(q_i))$ and $\mathbf{e}_s^i = \text{Embed}(\text{Mask}(s_i))$ denote the embedding vectors of the masked question and its corresponding SQL, respectively; $\text{sim}(\cdot, \cdot)$ is the cosine similarity function; and τ is a temperature parameter.

In our implementation, we adopt the Qwen3-Embedding architecture [46], employing a causal-attention-based LLM as the embedding encoder. Both the knowledge base and the training data for the embedding model are derived from the SQLFLOW dataset we constructed. The input format concatenates the task instruction with the masked question as follows: $\{\texttt{Instruction}\}$ $\{\texttt{Mask}(q_i)\}$ followed by an [EOS] token. The final sentence embedding is obtained from the hidden state of the [EOS] token in the last layer of the encoder. Our method retrieves few-shot examples that are highly aligned with the target query in SQL structure. This enhances the few-shot reasoning capability of closed-source LLMs on the Text-to-SQL task.

IV. EXPERIMENTS

A. Experimental Setup

1) Benchmarks: Multiple benchmarks are used to evaluate the effectiveness of our method.

- **Spider** [16] contains 10,181 NL questions and 5,693 unique SQL queries, involving 200 databases across multiple tables covering 138 different domains. We utilize both its development set, containing 1,034 samples, and its test set with 2,147 samples for evaluation.
- **BIRD** [17] includes 12,751 NL question-SQL pairs and 95 databases across 37 domains, focusing on large-scale noisy data and external knowledge reasoning. Our experiments are performed on its development set, which consists of 1,534 data samples.
- **EHRSQL** [45] includes approximately 24,000 NL question-SQL pairs linked to 2 open-source electronic health record databases. The dataset introduces challenges such as complex, time-sensitive questions and the inclusion of unanswerable questions.
- **Spider-DK, Spider-Syn, and Spider-Realistic** [47]–[49] are three widely-adopted robustness benchmarks designed to evaluate model performance in practical Text-to-SQL applications. Spider-DK tests the model's ability to understand implicit domain knowledge in NL questions, offering 535 samples for evaluation. Spider-Syn replaces schema-related words with manually selected synonyms and provides 1,034 evaluation samples. Spider-Realistic removes or paraphrases explicit mentions of column names, offering 508 samples for evaluation.

2) Evaluation Metric: Execution accuracy (EX) is adopted in our experiments. This metric compares the execution result of the predicted SQL query against that of the ground-truth SQL query on a live database instance.

3) Baselines of LLM Training: Multiple open-source and closed-source mainstream LLMs are evaluated as reference baselines. The closed-source models include GPT-4-Turbo [50] and GPT-4o [51]. The open-source models encompass DeepSeek-Coder-7B-Instruct [52], Qwen2.5-Coder-7B-Instruct [53], OpenCoder-8B-Instruct [54], Meta-Llama-3.1-8B-Instruct [55], and Granite-8B-Code-Instruct [56]. Regarding the training data for the models, we utilize not only the public datasets Spider-train [16] and BIRD-train [17], but also the synthetic dataset SynSQL proposed by OmniSQL [5]. We randomly sample a subset of SynSQL containing 75K examples (75,386 instances), matching the exact size of the SQLFLOW dataset for ease of comparison. In addition, we use the full SynSQL-2.5M dataset, comprising 2.5 million examples, as a reference. We use GPT-4o as the backend model to construct SQLFLOW and set the temperature to zero.

4) Baselines of Few-shot Example Retrieval: We compare our retrieval method with multiple example retrieval strategies under both masked and unmasked settings. The masking strategy in DAIL-SQL [6] is applied here. All similarity calculations use cosine similarity. Strategies are as follows:

TABLE II: Performance of LLMs on mainstream benchmarks. The first two blocks list closed-source and open-source base models, respectively, which are included as reference baselines. The last two blocks present fine-tuned models, where the first column indicates the training data setting."Seed Data" denotes the union of Spider-train, BIRD-train, and EHRSQL-train.

LLM / Training Data	Spider dev		Spider test		BIRD dev		EHRSQL		Spider-DK		Spider-Syn		Spider-Realistic		Average	
	Gre	Maj	Gre	Maj	Gre	Maj	Gre	Maj	Gre	Maj	Gre	Maj	Gre	Maj	Gre	Maj
Closed-source LLMs																
GPT-4-Turbo	72.4	72.2	83.4	84.2	62.0	63.6	43.1	44.8	72.3	72.1	62.9	63.5	67.5	68.3	66.2	67.0
GPT-4o	70.9	70.7	83.2	84.9	61.9	64.0	44.9	45.5	72.9	73.5	59.6	62.3	66.5	66.7	65.7	66.8
Open-source LLMs																
DeepSeek-Coder-7B-Instruct	63.2	63.2	70.5	73.2	43.1	48.0	28.6	33.9	60.9	64.1	49.9	51.7	58.7	58.9	53.6	56.1
Qwen2.5-Coder-7B-Instruct	73.4	77.1	82.2	85.6	50.9	61.3	24.3	36.9	67.5	73.6	63.1	66.9	66.7	70.5	61.2	67.4
OpenCoder-8B-Instruct	59.5	59.5	68.3	70.1	37.5	45.3	21.9	29.9	62.6	64.7	46.0	46.1	49.0	49.4	49.3	52.1
Meta-Llama-3.1-8B-Instruct	61.8	67.7	72.2	78.5	42.0	53.1	24.6	33.7	62.6	69.9	53.1	59.3	57.5	61.0	53.4	60.5
Granite-8B-Code-Instruct	58.5	59.2	64.9	68.6	27.6	32.5	16.0	22.6	50.7	54.4	45.0	46.8	48.8	49.4	44.5	47.6
Trained on Meta-Llama-3.1-8B-Instruct																
SynSQL(75k)	78.1	80.9	78.1	81.8	47.7	54.9	28.9	35.9	66.4	67.5	67.5	72.1	74.0	77.0	63.0	67.2
SynSQL(2.5M)	81.2	81.6	87.9	88.3	63.9	66.1	34.9	40.0	76.1	77.8	69.7	69.6	76.2	78.0	70.0	71.6
Seed Data + SQLFLOW	81.4	85.4	81.9	85.2	54.5	59.5	61.2	61.5	67.5	72.7	71.2	76.5	78.0	80.1	70.8	74.4
SQLFLOW	80.4	83.3	81.3	83.9	51.0	57.1	51.2	56.0	66.0	72.0	69.3	74.0	75.4	79.7	67.8	72.3
Trained on Qwen2.5-Coder-7B-Instruct																
SynSQL(75k)	80.7	83.6	81.6	83.7	53.2	59.8	34.6	43.8	68.4	67.9	68.8	71.9	75.8	79.5	66.2	70.0
SynSQL(2.5M)	81.2	81.6	87.9	88.3	63.9	66.1	34.9	40.0	76.1	77.8	69.7	69.6	76.2	78.0	70.0	71.6
Seed Data + SQLFLOW	84.9	86.2	85.3	87.3	58.0	62.1	61.2	62.0	74.4	75.9	74.8	78.0	82.1	83.5	74.4	76.4
SQLFLOW	83.4	85.0	85.1	86.6	56.8	61.0	53.7	57.5	73.3	74.8	73.4	76.4	81.9	81.7	72.5	74.7

- **Zero-shot** [57] refers to a setting in which no few-shot examples are used during inference.
- **Random Selection** means selecting the examples from the knowledge base randomly without any criteria.
- **Question Similarity Selection** [14] retrieves examples by selecting candidate questions with the highest similarity to the target question.
- **DAIL Selection** [6] retrieves examples by jointly considering question- and query-level similarity. Domain-specific tokens in questions are first masked, after which candidates are ranked by question similarity and filtered based on whether their query Jaccard similarity to a pre-generated reference SQL exceeds a predefined threshold.
- **Question-SQL Similarity Selection** encodes the target question and each candidate SQL into a shared embedding space with the untrained model, and selects the examples with the highest embedding similarity.
- **PreSQL-SQL Similarity Selection** ranks examples by the embedding similarity between each candidate SQL and a pre-generated SQL for the target question, and selects the most similar ones.
- **Question-AST Similarity Selection** parses each candidate SQL into an abstract syntax tree (AST) with `SQLGlot` and converts it into a compact, structure-preserving representation, which keeps all child expressions to retain query structure, while pruning semantically redundant fields (e.g., None arguments and default flags). Candidates are then ranked by the embedding similarity between the target question and the AST representation.

5) Implementation Details: To validate the effectiveness of our framework, we conduct full-parameter SFT on two widely adopted open-source LLMs: Meta-Llama-3.1-8B-Instruct [55] and Qwen2.5-Coder-7B-Instruct [53]. We use the entire SQLFLOW dataset as training data. The training configuration includes a learning rate of 2×10^{-5}, 2 training epochs, and a warmup ratio of 15%. During LLM inference, we explore two decoding strategies: greedy decoding and majority voting. Greedy decoding (denoted as `Gre` in tables) uses a temperature of 0 to produce deterministic outputs. Majority voting (denoted as `Maj` in tables) samples 8 candidate responses per input at a temperature of 0.8. From these candidates, valid SQL queries are extracted and executed; the final prediction is selected as the query whose execution result receives the most votes.

As for the retrieval module, we fine-tune a Qwen3-Embedding-0.6B model [46] via SFT. To construct the training data, we use a filtered subset of the SQLFlow-Spider and SQLFlow-BIRD datasets (described in Section III-B7), retaining only SQL queries that begin with a `SELECT` clause. This process yields 67,570 training instances, which we refer to as SQLFLOW-PART. The embedding model is trained for three epochs with a learning rate of 6×10^{-6} and a per-device batch size of 2. During training, the NL question is treated as the query, and its corresponding SQL query is used as the positive example. For each query, 30 negative SQL examples are randomly sampled from the remaining dataset. At inference time, we compute question–SQL similarity using cosine similarity between embeddings generated by both the original and the fine-tuned embedding models. We apply the

TABLE III: Performance comparison of retrieval strategies. "Upper limit": DAIL selection retrieval using ground-truth SQL. "API" indicates whether LLM API calls are required during retrieval, with an average token cost of 517 tokens per query.

Retrieval Strategy	API	BIRD dev	Spider dev	Spider test	Spider-DK	Spider-Realistic	Spider-Syn	Average
Reference strategies without mask								
Zero-shot	-	52.5	74.1	75.5	60.2	71.1	62.2	65.9
Random selection	✘	56.6	78.3	79.8	64.5	72.2	66.6	69.7
Question similarity	✘	58.3	79.6	81.9	66.2	74.4	67.7	71.4
DAIL selection	✔	58.8	80.0	<u>82.2</u>	66.4	75.2	68.9	71.9
PreSQL-SQL similarity	✔	59.1	78.9	81.0	63.7	74.2	69.6	71.1
Question-AST similarity	✘	58.2	79.6	80.7	64.5	73.6	65.7	70.4
Question-SQL similarity	✘	58.7	80.9	81.7	65.6	74.2	66.4	71.3
Upper limit	✔	**61.8**	**83.9**	**85.7**	**70.8**	**77.4**	**72.7**	**75.4**
SQLFlow-part+Spider+BIRD AST	✘	59.1	80.7	81.9	66.4	75.4	67.7	71.9
SQLFlow-part+Spider+BIRD SQL	✘	<u>59.3</u>	<u>81.0</u>	82.1	<u>67.3</u>	<u>75.6</u>	<u>70.1</u>	<u>72.6</u>
Reference strategies with mask								
Question similarity	✘	59.0	81.0	83.1	67.5	75.6	69.9	72.7
DAIL selection	✔	59.4	80.8	82.9	67.1	75.2	69.8	72.5
PreSQL-SQL similarity	✔	59.3	77.5	78.5	63.6	71.9	67.5	69.7
Question-AST similarity	✘	56.8	78.2	79.6	65.4	73.0	67.2	70.0
Question-SQL similarity	✘	57.8	80.1	80.3	65.6	73.4	69.0	71.0
Upper limit	✔	**61.6**	**83.8**	**85.1**	**69.2**	**78.7**	**73.6**	**75.3**
SQLFlow-part+Spider+BIRD AST	✘	59.5	<u>81.3</u>	83.0	<u>68.8</u>	76.4	70.9	73.3
SQLFlow-part+Spider+BIRD SQL	✘	<u>61.0</u>	81.2	<u>83.5</u>	67.7	<u>77.2</u>	<u>71.1</u>	<u>73.6</u>

masking strategy proposed in DAIL–SQL [6] during both training and inference. Retrieved examples are incorporated into the prompt using a 5-shot setting for LLM inference. For experiments involving closed-source LLMs, we use GPT-4o with greedy decoding and set the temperature to zero. Prompt construction follows the organization strategy introduced in DAIL. In addition to SQLFLOW-PART, we construct two baseline datasets for comparison. The first baseline combines the standard Spider-train and BIRD-train splits. The second baseline, denoted as SYNSQL-PART, consists of 67,570 SQL queries randomly sampled from SynSQL-2.5M [5], matched in size with SQLFLOW-PART to ensure a fair comparison.

All experiments are conducted on an Ubuntu 22.04.4 LTS server equipped with eight NVIDIA H200 GPUs. LLM SFT is implemented using Llama-Factory v0.9.3, while retrieval model SFT is carried out with SWIFT v3.6.3. Inference for both models is handled by vLLM v0.10.0.

B. Performance of LLM Fine-tuning

As shown in Table II, the data generated by our framework leads to consistent performance improvements across multiple mainstream benchmarks, demonstrating the effectiveness of TEXT2SQL-FLOW. We fine-tune two widely used open-source models, Meta-Llama-3.1-8B-Instruct and Qwen2.5-Coder-7B-Instruct, to evaluate the general applicability of our data. In both cases, models trained on our generated data significantly outperformed their respective baselines as well as other models. When fine-tuned with SQLFLOW data, Qwen2.5-Coder-7B-Instruct achieved notable gains: execution accuracy ("Gre") on Spider-dev increased from 80.7% to 83.4% (+2.7%), on BIRD-dev from 53.2% to 56.8% (+3.6%), and on the challenging EHRSQL benchmark from 34.6% to 53.7% (+19.1%). These results confirm two key points: (1) SFT substantially enhances model performance on Text-to-SQL tasks, and (2) the data generated via our TEXT2SQL-FLOW framework exhibits high quality and strong training utility. The consistent improvements across two distinct model architectures demonstrate the broad effectiveness of our dataset in boosting model performance.

In comparison with other training datasets, our data demonstrates clear advantages. Specifically, models trained with SQLFLOW (75K) not only outperform the baseline models but also consistently surpass SynSQL (75K), highlighting the robustness of our data generation approach. Remarkably, despite being trained on a substantially smaller dataset, the model fine-tuned with SQLFLOW (75K) achieves performance comparable to SynSQL-2.5M on several challenging benchmarks. Furthermore, when real-world seed data, including Spider-train, BIRD-train, and EHRSQL-train, are incorporated into the training set, the resulting model attains state-of-the-art (SOTA) performance across multiple benchmarks. These results indicate that expanding seed data with SQLFLOW effectively enhances model performance.

TABLE IV: Comparison of training data constructed under different ablation settings. # Tokens per SQL indicates the average tokens consumed to generate each augmented SQL.

Training Setting	Spider dev	Spider test	BIRD dev	# Token per SQL
Baseline	78.3	79.8	52.5	-
Full Pipeline	81.7	83.2	55.5	4909
Auxiliary Database Selection				
w/o Auxiliary Database	81.1	82.3	54.8	0
Random Selection	80.9	81.7	54.0	
SQL Augmentation Strategy				
w/o DVT	80.9	82.5	54.0	857
w/o QSM	80.1	81.8	53.4	
w/o BLC	79.7	82.3	54.0	
w/o CE	79.7	81.9	53.8	
w/o ASF	81.1	81.5	55.5	
w/o PO	79.9	81.7	54.2	
w/o Executability Filter	80.1	82.3	54.1	0
Question Style				
w/o TF	80.4	83.1	53.5	1742
w/o SSI	80.1	82.0	55.4	
w/o IDC	80.9	82.3	54.0	
w/o IP	81.5	82.8	54.0	
w/o Correspondence Filter	80.5	80.0	53.4	709
w/o Prompt Generation	-	-	-	0
w/o Chain-of-Thought	75.5	78.1	53.3	1601

C. Ablation Study of the Text2SQL-Flow Pipeline

We conduct an ablation study to analyze the contribution of each component in TEXT2SQL-FLOW. Results are reported in Table IV. We select 20 databases from Spider as original databases and 20 databases from BIRD as auxiliary databases, from which 200 representative SQLs are sampled per source using SQL-structure-based bucketing, resulting in 400 seed SQLs. Each seed SQL is augmented using all six SQL augmentation strategies, each instantiated twice, yielding $4,800$ augmented SQLs. Each augmented SQL is paired with four question styles, producing $19{,}200$ SQL–question pairs. All components of our pipeline are enabled unless explicitly ablated. The Baseline (line 1) is trained solely on the seed SQLs, whereas all other settings are trained on augmented data with a fixed size of 2,400 instances. The Full Pipeline (line 2) samples training data evenly from Spider and BIRD. w/o Auxiliary Database (line 3) draws samples exclusively from the Spider source while keeping the total number of sampled instances fixed at 2,400, whereas Random Database Selection (line 4) replaces the structured auxiliary database selection strategy with random sampling. For SQL augmentation and question-style ablations, one strategy or style is removed at a time, with uniform sampling over the remaining ones; the corresponding category names follow the abbreviations defined in Section III-B1 and Section III-B3. In w/o Executability Filter (line 11) and w/o Correspondence Filter (line 16), invalid or misaligned samples are retained during training. w/o Chain-of-Thought (line 18) removes the CoT rationales while keeping the same training samples.

Full Pipeline consistently outperforms the baseline, demonstrating the effectiveness of TEXT2SQL-FLOW. Removing auxiliary databases degrades performance, while random database selection further harms results, validating the necessity of auxiliary databases and our structured coverage strategy. Ablating any SQL augmentation strategy or question style leads to performance drops, indicating that both SQL-level diversity and linguistic variation are important. Disabling executability or alignment filters consistently hurts performance, highlighting the importance of data quality control. Removing CoT supervision causes the most severe degradation, underscoring the critical role of explicit reasoning signals.

TABLE V: Performance of models trained on the same amount of synthetic data generated by different LLMs. "API cost" is reported in USD per million tokens and is set to zero for open-source models, excluding local compute overhead.

LLM Backbone	API Cost	Spider dev	Spider test	BIRD dev
Baseline	-	78.7	78.0	50.3
Closed-source LLMs				
Claude-3.7-Sonnet	12	80.8	83.3	50.9
Gemini-2.5-Flash	0.6	82.8	83.0	52.3
GPT-3.5-Turbo	0.5	80.9	81.7	51.6
GPT-4o	2.5	81.2	81.7	54.1
Open-source LLMs				
DeepSeek-V3	0	79.8	81.0	50.7
GLM-4.6	0	80.9	81.0	51.4
Qwen-3-235B	0	80.5	80.6	50.8

D. Robustness to the Choice of Data Generation Models

Many components of TEXT2SQL-FLOW rely on LLMs, making the choice of the data generation backbone a potentially influential factor for the quality of the augmented data. To examine this sensitivity, we instantiate the same SQL data augmentation pipeline with a diverse set of both closed-source and open-source LLMs. Throughout this study, we keep the overall pipeline design, augmentation procedures, and the scale of synthetic data fixed, and vary only the LLM used for data generation. Specifically, each model is prompted to generate 1,000 synthetic training examples based on 500 samples randomly selected from the Spider training set. The resulting augmented data are then used to train the downstream Text-to-SQL model. Baseline (line 1) refers to the model trained using data constructed solely from these 500 samples.

As shown in Table V, the downstream performance remains competitive across a wide range of data generation models. While stronger LLMs tend to produce slightly better synthetic data, even relatively weaker or low-cost models, including GPT-3.5-Turbo and open-source LLMs, still yield solid performance on benchmarks. This suggests that TEXT2SQL-FLOW does not critically depend on highly sophisticated LLM reasoning during data generation. Instead, its robustness primarily stems from the structured augmentation process, executability filtering, and semantic alignment, which together mitigate

noise introduced by imperfect generation models. Across all LLM backbones, the performance variation remains within a moderate range, with all models outperforming the baseline, without altering the overall effectiveness of the pipeline.

E. Comparison of Retrieval Strategies

We compare our retrieval method with several baselines, where the embedding model is trained on SQLFlow-part, Spider-train, and BIRD-train, using Qwen3-Embedding-0.6B as the base model. As shown in the lower part of Table III, fine-tuning the retrieval model on high-quality synthetic data from SQLFlow-part using our retrieval strategy significantly boosts performance. Both the high-quality augmented data and the masked alignment retrieval method contribute to this improvement. Specifically, our Question-SQL alignment method without masking achieves 59.3%, 81.0%, and 82.1% accuracy on BIRD-dev, Spider-dev, and Spider-test, respectively, outperforming the baseline that relies on Question-SQL similarity retrieval with the base (non-finetuned, no mask) model. This indicates that fine-tuning with augmented data enables the model to select more relevant examples, thereby enhancing the in-context learning capability of downstream closed-source models for accurate SQL generation.

The masking strategy is another key contributor to this improvement. Ablation studies confirm its effectiveness: for non-finetuned methods, masking consistently improves retrieval performance (e.g., DAIL selection accuracy on BIRD-dev increases from 58.8% to 59.4%). Similarly, when we fine-tune on SQLFlow-part and use masking during training, performance improves to 61.0%, 81.2%, and 83.5% on the three benchmarks. By masking literal values in the training data, the model learns to focus on the structural correspondence between questions and SQL queries while ignoring superficial lexical differences. This enables it to retrieve examples whose SQL patterns closely match the unseen ground-truth query based solely on the input question in a single step. We also evaluate ASTs as an alternative SQL representation for embedding-model training and example retrieval in our Question-AST similarity setting. However, converting SQL into ASTs can obscure semantic cues but may introduce additional noise, leading to fewer improvements on some benchmarks. Since SQL already encodes rich structural information, our Question-SQL alignment is designed to enable the retrieval model to learn question–query correspondences in a data-driven manner while preserving the full SQL signal. In contrast, strategies such as DAIL Selection and PreSQL-SQL Selection require generating an intermediate SQL query for retrieval, which incurs additional API costs and risks propagating errors from intermediate generations.

F. Comparison of Training Data for Retrieval Model

As shown in Figure 5, the model trained solely on the BIRD dataset performs well on BIRD-dev but suffers a sharp performance drop on Spider-dev and Spider-test, indicating limited cross-benchmark generalization when training on a single dataset. However, simply mixing data from multiple sources

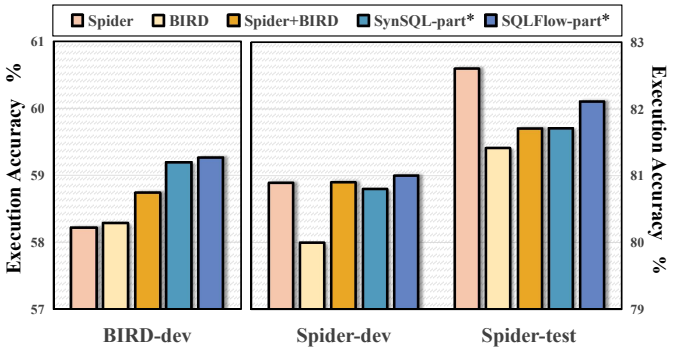

Fig. 5: Performance of question–SQL alignment strategy retrieval models trained on different datasets. No masking operations are applied during the retrieval process. "SynSQL-part*" and "SQLFlow-part*" denote the corresponding datasets combined with Spider-train and BIRD-train.

does not necessarily enhance generalization. For example, training on both Spider and BIRD data for the Spider benchmark even leads to degraded performance. In contrast, the model trained on the SQLFlow-part combined with Spider and BIRD achieves better results, effectively mitigating overfitting and substantially improving cross-benchmark performance. This indicates that the benefit does not stem merely from larger data volume, but from the diversity and quality of the data generated by TEXT2SQL-FLOW, which more effectively boosts model generalization. A comparison with SynSQL-part further validates this conclusion. As a fully synthetic dataset, SynSQL exhibits a data distribution that diverges significantly from real-world applications. Although it aims for generalization, its effectiveness on specific tasks remains limited, demanding massive data and computation for only modest gains. For instance, SynSQL-part achieves only slight improvement on the Spider benchmark, barely exceeding the Spider+BIRD combination. In contrast, SQLFlow-part delivers markedly better results, highlighting its superior data quality, distribution alignment, and task relevance.

V. CONCLUSION

In this work, we introduced TEXT2SQL-FLOW, a robust SQL-aware data augmentation framework that enables the scalable generation of high-quality, structurally diverse Text-to-SQL examples from limited seed data. Leveraging this framework, we constructed SQLFLOW, a large-scale dataset comprising 75,386 annotated instances. Experimental results show that models fine-tuned on SQLFLOW consistently outperform those trained on existing comparable datasets, highlighting the critical role of data quality and structural diversity in enhancing model generalization and compositional reasoning. Furthermore, our proposed masked alignment retrieval method improves the in-context learning performance of closed-source large language models by enabling structure-aware example selection. This work establishes a data-centric foundation for future Text-to-SQL research and demonstrates the importance of high-quality data in data-centric AI.

REFERENCES

[1] D. Zha, Z. P. Bhat, K.-H. Lai, F. Yang, Z. Jiang, S. Zhong, and X. Hu, "Data-centric artificial intelligence: A survey," *ACM Computing Surveys*, vol. 57, no. 5, pp. 1–42, 2025.

[2] J. Jakubik, M. Vössing, N. Kühl, J. Walk, and G. Satzger, "Data-centric artificial intelligence," *Business & Information Systems Engineering*, vol. 66, no. 4, pp. 507–515, 2024.

[3] M. Haukkala, "Data quality in artificial intelligence," 2022.

[4] A. B. Kanburoğlu and F. B. Tek, "Text-to-sql: A methodical review of challenges and models," *Turkish Journal of Electrical Engineering and Computer Sciences*, vol. 32, no. 3, pp. 403–419, 2024.

[5] H. Li, S. Wu, X. Zhang, X. Huang, J. Zhang, F. Jiang, S. Wang, T. Zhang, J. Chen, R. Shi *et al.*, "Omnisql: Synthesizing high-quality text-to-sql data at scale," *arXiv preprint arXiv:2503.02240*, 2025.

[6] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, and J. Zhou, "Text-to-sql empowered by large language models: A benchmark evaluation," *arXiv preprint arXiv:2308.15363*, 2023.

[7] Y. Gao, Y. Liu, X. Li, X. Shi, Y. Zhu, Y. Wang, S. Li, W. Li, Y. Hong, Z. Luo *et al.*, "Xiyan-sql: A multi-generator ensemble framework for text-to-sql," *arXiv preprint arXiv:2411.08599*, 2024.

[8] H. Li, J. Zhang, H. Liu, J. Fan, X. Zhang, J. Zhu, R. Wei, H. Pan, C. Li, and H. Chen, "Codes: Towards building open-source language models for text-to-sql," *Proceedings of the ACM on Management of Data*, vol. 2, no. 3, pp. 1–28, 2024.

[9] L. Sheng and S.-S. Xu, "Slm-sql: An exploration of small language models for text-to-sql," *arXiv preprint arXiv:2507.22478*, 2025.

[10] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in neural information processing systems*, vol. 35, pp. 24 824–24 837, 2022.

[11] C.-Y. Tai, Z. Chen, T. Zhang, X. Deng, and H. Sun, "Exploring chain-of-thought style prompting for text-to-sql," *arXiv preprint arXiv:2305.14215*, 2023.

[12] H. Li, J. Zhang, C. Li, and H. Chen, "Resdsql: Decoupling schema linking and skeleton parsing for text-to-sql," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 11, 2023, pp. 13 067–13 075.

[13] M. Pourreza and D. Rafiei, "Din-sql: Decomposed in-context learning of text-to-sql with self-correction," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[14] A. Liu, X. Hu, L. Wen, and P. S. Yu, "A comprehensive evaluation of chatgpt's zero-shot text-to-sql capability," *arXiv preprint arXiv:2303.13547*, 2023.

[15] L. Nan, Y. Zhao, W. Zou, N. Ri, J. Tae, E. Zhang, A. Cohan, and D. Radev, "Enhancing few-shot text-to-sql capabilities of large language models: A study on prompt design strategies," *arXiv preprint arXiv:2305.12586*, 2023.

[16] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman *et al.*, "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task," *arXiv preprint arXiv:1809.08887*, 2018.

[17] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Geng, N. Huo *et al.*, "Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[18] J. Yang, B. Hui, M. Yang, J. Yang, J. Lin, and C. Zhou, "Synthesizing text-to-sql data from weak and strong llms," *arXiv preprint arXiv:2408.03256*, 2024.

[19] F. Li and H. V. Jagadish, "Constructing an interactive natural language interface for relational databases," *Proceedings of the VLDB Endowment*, vol. 8, no. 1, pp. 73–84, 2014.

[20] T. Mahmud, K. A. Hasan, M. Ahmed, and T. H. C. Chak, "A rule based approach for nlp based query processing," in *2015 2nd international conference on electrical information and communication technologies (EICT)*. IEEE, 2015, pp. 78–82.

[21] J. M. Zelle and R. J. Mooney, "Learning to parse database queries using inductive logic programming," in *Proceedings of the national conference on artificial intelligence*, 1996, pp. 1050–1055.

[22] V. Zhong, C. Xiong, and R. Socher, "Seq2sql: Generating structured queries from natural language using reinforcement learning," *arXiv preprint arXiv:1709.00103*, 2017.

[23] J. Guo, Z. Zhan, Y. Gao, Y. Xiao, J.-G. Lou, T. Liu, and D. Zhang, "Towards complex text-to-sql in cross-domain database with intermediate representation," *arXiv preprint arXiv:1905.08205*, 2019.

[24] B. Wang, R. Shin, X. Liu, O. Polozov, and M. Richardson, "Rat-sql: Relation-aware schema encoding and linking for text-to-sql parsers," *arXiv preprint arXiv:1911.04942*, 2019.

[25] Z. Chen, L. Chen, Y. Zhao, R. Cao, Z. Xu, S. Zhu, and K. Yu, "Shadowgnn: Graph projection neural network for text-to-sql parser," *arXiv preprint arXiv:2104.04689*, 2021.

[26] J. Devlin, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.

[27] P. Yin, G. Neubig, W.-t. Yih, and S. Riedel, "Tabert: Pretraining for joint understanding of textual and tabular data," *arXiv preprint arXiv:2005.08314*, 2020.

[28] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of machine learning research*, vol. 21, no. 140, pp. 1–67, 2020.

[29] Y. Fu, W. Ou, Z. Yu, and Y. Lin, "Miga: a unified multi-task generation framework for conversational text-to-sql," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 11, 2023, pp. 12 790–12 798.

[30] Z. Gu, J. Fan, N. Tang, L. Cao, B. Jia, S. Madden, and X. Du, "Few-shot text-to-sql translation using structure and content prompt learning," *Proceedings of the ACM on Management of Data*, vol. 1, no. 2, pp. 1–28, 2023.

[31] X. Dong, C. Zhang, Y. Ge, Y. Mao, Y. Gao, J. Lin, D. Lou *et al.*, "C3: Zero-shot text-to-sql with chatgpt," *arXiv preprint arXiv:2307.07306*, 2023.

[32] M. M. H. Nahid, D. Rafiei, W. Zhang, and Y. Zhang, "Rethinking schema linking: A context-aware bidirectional retrieval approach for text-to-sql," *arXiv preprint arXiv:2510.14296*, 2025.

[33] Z. Cao, Y. Zheng, Z. Fan, X. Zhang, W. Chen, and X. Bai, "Rsl-sql: Robust schema linking in text-to-sql generation," *arXiv preprint arXiv:2411.00073*, 2024.

[34] C. Li, Y. Wang, Z. Wu, Z. Yu, F. Zhao, S. Huang, and X. Dai, "Multisql: A schema-integrated context-dependent text2sql dataset with diverse sql operations," in *Findings of the Association for Computational Linguistics ACL 2024*, 2024, pp. 13 857–13 867.

[35] M. Pourreza, S. Talaei, R. Sun, X. Wan, H. Li, A. Mirhoseini, A. Saberi, S. Arik *et al.*, "Reasoning-sql: Reinforcement learning with sql tailored partial rewards for reasoning-enhanced text-to-sql," *arXiv preprint arXiv:2503.23157*, 2025.

[36] M. Glass, M. Eyceoz, D. Subramanian, G. Rossiello, L. Vu, and A. Gliozzo, "Extractive schema linking for text-to-sql," *arXiv preprint arXiv:2501.17174*, 2025.

[37] M. Kothyari, D. Dhingra, S. Sarawagi, and S. Chakrabarti, "Crush4sql: Collective retrieval using schema hallucination for text2sql," *arXiv preprint arXiv:2311.01173*, 2023.

[38] D. Guo, Y. Sun, D. Tang, N. Duan, J. Yin, H. Chi, J. Cao, P. Chen, and M. Zhou, "Question generation from sql queries improves neural semantic parsing," *arXiv preprint arXiv:1808.06304*, 2018.

[39] B. Wang, W. Yin, X. V. Lin, and C. Xiong, "Learning to synthesize data for semantic parsing," *arXiv preprint arXiv:2104.05827*, 2021.

[40] K. Wu, L. Wang, Z. Li, A. Zhang, X. Xiao, H. Wu, M. Zhang, and H. Wang, "Data augmentation with hierarchical sql-to-question generation for cross-domain text-to-sql parsing," *arXiv preprint arXiv:2103.02227*, 2021.

[41] V. Zhong, M. Lewis, S. I. Wang, and L. Zettlemoyer, "Grounded adaptation for zero-shot executable semantic parsing," *arXiv preprint arXiv:2009.07396*, 2020.

[42] W. Yang, P. Xu, and Y. Cao, "Hierarchical neural data synthesis for semantic parsing," *arXiv preprint arXiv:2112.02212*, 2021.

[43] T. Yu, C.-S. Wu, X. V. Lin, B. Wang, Y. C. Tan, X. Yang, D. Radev, R. Socher, and C. Xiong, "Grappa: Grammar-augmented pre-training for table semantic parsing," *arXiv preprint arXiv:2009.13845*, 2020.

[44] T. Yu, M. Yasunaga, K. Yang, R. Zhang, D. Wang, Z. Li, and D. Radev, "Syntaxsqlnet: Syntax tree networks for complex and cross-domaintext-to-sql task," *arXiv preprint arXiv:1810.05237*, 2018.

[45] G. Lee, H. Hwang, S. Bae, Y. Kwon, W. Shin, S. Yang, M. Seo, J.-Y. Kim, and E. Choi, "Ehrsql: A practical text-to-sql benchmark for electronic health records," *Advances in Neural Information Processing Systems*, vol. 35, pp. 15 589–15 601, 2022.

[46] Y. Zhang, M. Li, D. Long, X. Zhang, H. Lin, B. Yang, P. Xie, A. Yang, D. Liu, J. Lin *et al.*, "Qwen3 embedding: Advancing text embedding and reranking through foundation models," *arXiv preprint arXiv:2506.05176*, 2025.

[47] Y. Gan, X. Chen, and M. Purver, "Exploring underexplored limitations of cross-domain text-to-sql generalization," *arXiv preprint arXiv:2109.05157*, 2021.

[48] Y. Gan, X. Chen, Q. Huang, M. Purver, J. R. Woodward, J. Xie, and P. Huang, "Towards robustness of text-to-sql models against synonym substitution," *arXiv preprint arXiv:2106.01065*, 2021.

[49] X. Deng, A. H. Awadallah, C. Meek, O. Polozov, H. Sun, and M. Richardson, "Structure-grounded pretraining for text-to-sql," *arXiv preprint arXiv:2010.12773*, 2020.

[50] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[51] A. Hurst, A. Lerer, A. P. Goucher, A. Perelman, A. Ramesh, A. Clark, A. Ostrow, A. Welihinda, A. Hayes, A. Radford *et al.*, "Gpt-4o system card," *arXiv preprint arXiv:2410.21276*, 2024.

[52] D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. Li *et al.*, "Deepseek-coder: When the large language model meets programming–the rise of code intelligence," *arXiv preprint arXiv:2401.14196*, 2024.

[53] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu *et al.*, "Qwen2. 5-coder technical report," *arXiv preprint arXiv:2409.12186*, 2024.

[54] S. Huang, T. Cheng, J. K. Liu, J. Hao, L. Song, Y. Xu, J. Yang, J. Liu, C. Zhang, L. Chai *et al.*, "Opencoder: The open cookbook for top-tier code large language models," *arXiv preprint arXiv:2411.04905*, 2024.

[55] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[56] M. Mishra, M. Stallone, G. Zhang, Y. Shen, A. Prasad, A. M. Soria, M. Merler, P. Selvam, S. Surendran, S. Singh *et al.*, "Granite code models: A family of open foundation models for code intelligence," *arXiv preprint arXiv:2405.04324*, 2024.

[57] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," *Advances in neural information processing systems*, vol. 35, pp. 22 199–22 213, 2022.

APPENDIX A
ADDITIONAL PIPELINE COMPONENTS

In practical applications, to facilitate the organization and management of data generated by the pipeline and to support downstream usage, we additionally introduce two classifier steps to categorize the generated SQL queries. These operators are primarily user-oriented and are designed to improve system usability and engineering practicality, rather than to directly enhance the model performance emphasized in this paper. For this reason, their detailed descriptions are deferred to the Appendix. Based on the resulting classifications, users can more conveniently and efficiently inspect, analyze, and utilize the generated data. We describe the two operators as follows.

A. *SQL Component Difficulty Classifier*

Classifying generated SQL queries facilitates a deeper analysis and understanding of their structural complexity. Following the evaluation criteria established in the Spider benchmark, we categorize SQL queries into four difficulty levels: *easy*, *medium*, *hard*, and *extra hard*. This classification is primarily based on the number and complexity of syntactic components present in the query, including column selections and the use of aggregate functions in the `SELECT` clause, as well as the presence of keywords such as `GROUP BY`, `ORDER BY`, `INTERSECT`, or advanced constructs like nested subqueries. Generally, queries incorporating a greater number of such components are considered structurally more complex and thus assigned a higher difficulty level. Under this principle, the augmented SQL query s_i^{aug} can be assigned a reasonable and quantifiable component-based difficulty. This process is formally expressed as:

$$cd_i = \text{CC}(s_i^{\text{aug}}) \tag{9}$$

where $\text{CC}(\cdot)$ denotes the mapping function of the classifier, and cd_i represents the component difficulty of the query.

B. *SQL Execution Difficulty Classifier*

It is important to note that structural complexity alone does not fully determine the overall difficulty of an SQL query: some queries with numerous components may be logically straightforward and thus easier for models to generate correctly. From the perspective of LLMs, a more meaningful measure of difficulty is whether the model can consistently produce the correct SQL for a given NL question in the Text-to-SQL task. To this end, we introduce the *execution difficulty* metric ed_i. Specifically, we prompt the LLM to perform the Text-to-SQL task k times for the same input prompt p_i and count the number of successful executions n. The execution difficulty is then defined as:

$$ed_i = 1 - \frac{n}{k} \tag{10}$$

This metric reflects the practical capability of the current LLM in solving a specific Text-to-SQL problem. Unlike the component-based classifier, execution difficulty is model-dependent. More capable LLMs will achieve higher success rates on the same task, thereby exhibiting lower execution difficulty. Consequently, this metric provides a dynamic and performance-oriented perspective on task difficulty, better aligned with real-world generation behavior.

C. *Pipeline Workflow*

In the complete pipeline, the two classifier operators are incorporated as integral components. For each generated SQL query, the SQL Component Classifier and the SQL Execution Classifier are applied to assign difficulty labels to the corresponding data sample. Through this process, the pipeline produces data samples in the following format:

$$\left\{s_i^{\text{aug}},\, q_i,\, S,\, p_i,\, \text{cot}_i,\, \text{ed}_i,\, \text{cd}_i\right\}_{i=1}^{N}$$

APPENDIX B
DETAILS OF DATABASE INTERACTION IMPLEMENTATION

Within the framework of TEXT2SQL-FLOW, a robust and efficient database interaction mechanism constitutes a core infrastructural component that underpins stable and scalable pipeline execution. Multiple critical stages of the pipeline, including SQL execution filtering and SQL augmentation, require frequent and reliable access to the underlying database system. These interactions can be broadly categorized into two types: (1) *SQL Execution* and (2) *Schema Metadata Extraction*.

SQL Execution refers to submitting generated SQL queries to the database engine and collecting execution feedback, which is primarily used to validate the executability of candidate queries. In practice, mainstream database systems such as SQLite, MySQL, and PostgreSQL expose heterogeneous driver interfaces and connection protocols. Directly relying on database-specific native APIs complicates cross-platform support and requires maintaining separate adaptation logic for each backend, leading to code redundancy and severely limiting system extensibility.

Schema Metadata Extraction aims to retrieve structured information about the database schema, including `CREATE TABLE` definitions, column attributes (e.g., data types, nullability, default values), primary and foreign key constraints, index metadata, and representative `INSERT` statements used for SQL augmentation. Such metadata is essential for understanding database semantics, generating syntactically and semantically valid SQL queries, and enabling context-aware data augmentation strategies. Although most relational databases expose schema information via mechanisms such as `INFORMATION_SCHEMA` or system catalog tables, their query syntax, naming conventions, and output formats vary substantially across systems. Moreover, for a given database instance, schema information typically remains stable over short time horizons. Repeatedly querying identical metadata therefore introduces unnecessary I/O overhead and latency, which can significantly degrade pipeline throughput at scale.

To address these challenges, we design and implement a `Database Manager` module as part of the data augmentation framework. This module abstracts low-level

database interaction details and exposes a unified, efficient, and extensible programming interface to upper-layer components. Specifically, the Database Manager supports `batch_sql_execution` and `batch_compare_sql`, enabling higher throughput under high-concurrency workloads. It also encapsulates schema metadata retrieval and provides high-level interfaces such as `get_ddl` (for Data Definition Language extraction) and `get_insert_statement` (for generating representative insertion statements), thereby decoupling upper-layer logic from database-specific handling.

To ensure robust cross-database compatibility, we introduce an abstract base class, `DatabaseConnector`, which defines a standardized interaction interface. This interface includes methods such as `connect_db` (for connection establishment and initialization), `execute_sql` (for executing arbitrary SQL statements), and `get_schema` (for extracting complete schema metadata). Concrete connectors for specific database systems (e.g., `SQLiteDatabaseConnector`) inherit from this base class and implement database-specific driver calls and error handling. At runtime, the Database Manager dynamically instantiates the appropriate connector based on configuration, while its core logic remains agnostic to the underlying database type. This design achieves the architectural principle of "design once, adapt to many databases."

To further improve throughput in large-scale SQL generation and validation scenarios, the Database Manager caches each database's structured schema representation in memory. Although individual metadata queries are relatively fast, the pipeline repeatedly accesses the same schema when generating and validating multiple SQL candidates per database, making the cumulative overhead non-trivial. Upon the first access to a database, its schema (including tables, columns, keys, and sampled rows) is extracted, serialized, and stored in an in-memory cache. Subsequent accesses are served directly from the cache, avoiding repeated schema reconstruction and redundant metadata queries.

In summary, the Database Manager decouples the pipeline from underlying database implementations, optimizes execution efficiency, and enhances system modularity. Its extensible, plugin-style architecture lays a solid foundation for supporting additional database types in the future.

APPENDIX C
CASE STUDY OF FEW-SHOT RETRIEVAL

To illustrate how our method retrieves superior examples and why these examples effectively guide the model toward generating correct SQL, we present a case study from the BIRD-dev dataset, which is shown in Figure 6. DAIL Selection's retrieval mechanism heavily relies on its pre-generated SQL. However, in this particular case, the pre-generated query is inaccurate both structurally and semantically. It fails to capture the key complex logic present in the Gold SQL, namely, returning only a single maximum-value result. Consequently, although the examples retrieved by DAIL Selection align structurally with this erroneous pre-generated query, they are entirely unrelated to the skeleton of the Gold SQL.

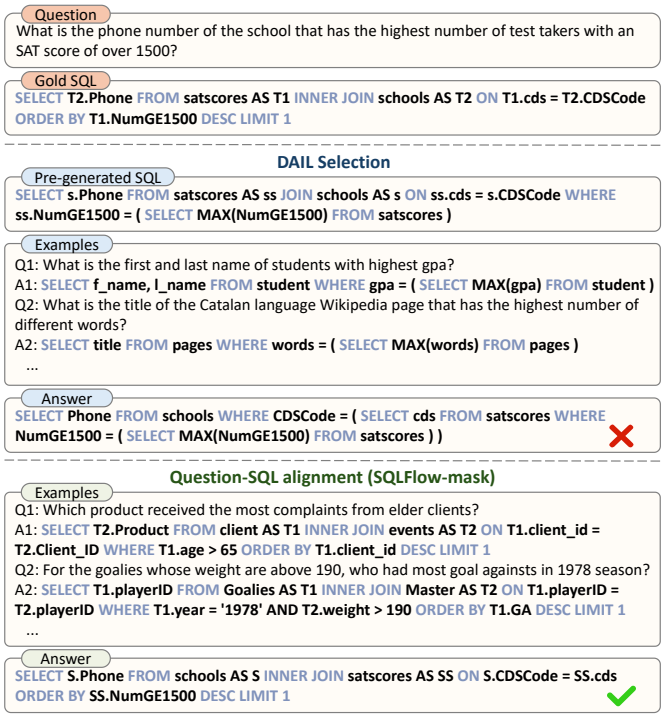

Fig. 6: Case study of few-shot retrieval.

These structurally flawed and irrelevant examples ultimately mislead the LLM, causing it to produce an incorrect answer. In contrast, our method directly aligns the question with the SQL skeleton, enabling it to retrieve examples highly similar to the Gold SQL skeleton using only the original question. As shown in the figure, the examples retrieved by our approach correctly include essential structural components such as `INNER JOIN`, `ORDER BY ... DESC`, and `LIMIT 1`. This demonstrates that our skeleton alignment mechanism captures not only syntactic similarity but also the core logical intent of the query. These high-quality examples provide the LLM with an accurate structural template, thereby guiding it to generate the correct SQL query.